

Практическая работа «Сжатие изображений в системе MATLAB»

Цель: Знакомство с алгоритмом JPEG в системе Matlab. Исследование эффективности JPEG-сжатия. Получение навыков реализации линейных ортогональных преобразований.

Содержание отчета: ответы на контрольные вопросы, выполненные задания и вывод о проделанной работе.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Сжатие изображений – основные понятия

Сжатие изображений основывается на их изначальной избыточности, что позволяет при ее устранении сократить объем данных, необходимых для представления цифрового изображения. Методы сжатия подразделяются на две категории [1]:

- сжатие без потерь – восстановленное изображение полностью идентично первоначальному;
- сжатие с потерями – восстановленное изображение не является идентичным первоначальному.

Ко второй категории относится формат сжатия *JPEG*, который будет рассмотрен в данной лабораторной работе. Данный формат сжатия основан на удалении визуальной избыточности, заключающейся в наличии в изображении информации, не воспринимаемой органами зрения человека [1]. Степень сжатия определяется по представленной ниже формуле:

$$K_{сж} = \frac{Im_1}{Im_2},$$

где Im_1 - объем исходного изображения в байтах, Im_2 - объем сжатого изображения в байтах, $k_{сж}$ – безразмерный коэффициент сжатия.

Для получения информации о размере изображения (в байтах), а также другие сведения о графическом файле используется функция *imfinfo*, синтаксис которой приведен ниже:

$$info = imfinfo(path),$$

где *path* – строковая переменная, содержащая путь к файлу изображения, *info* – структура, хранящая в себе сведения об изображении.

Пример:

```
info = imfinfo('F:/M/fishboat.png')
```

```
info =
```

```
Filename: 'F:/M/fishboat.png'  
FileModDate: '09-may-2018 13:20:19'  
FileSize: 177762  
Format: 'png'  
FormatVersion: []  
Width: 512  
Height: 512  
BitDepth: 8  
ColorType: 'grayscale'  
FormatSignature: [137 80 78 71 13 10 26 10]
```

```

Colormap: []
Histogram: []
InterlaceType: 'none'
Transparency: 'none'
...
XResolution: 47244
YResolution: 47244
ResolutionUnit: 'meter'

```

Как видно, структура `info` содержит ряд сведений об изображении (формат, разрешение, число битов на пиксель и т. д.). Поскольку список сведений весьма обширен, некоторые свойства не были отображены в данном примере.

Для получения конкретного свойства из данной структуры достаточно написать название данной структуры, поставить после нее точку, а после нее указать название необходимого свойства.

Например:

```

size = info.FileSize
size =
177762

```

Как видно из примера, файл `'F:/M/boat.png'` занимает объем 177762 байтов.

Сжатие по стандарту JPEG

Формат сжатия JPEG – самый популярный на сегодняшний день формат сжатия в сети Интернет, использование которого позволяет достичь высоких коэффициентов сжатия при относительно хорошем качестве изображения.

Упрощенная схема алгоритма сжатия JPEG представлена на рисунке 2.1.

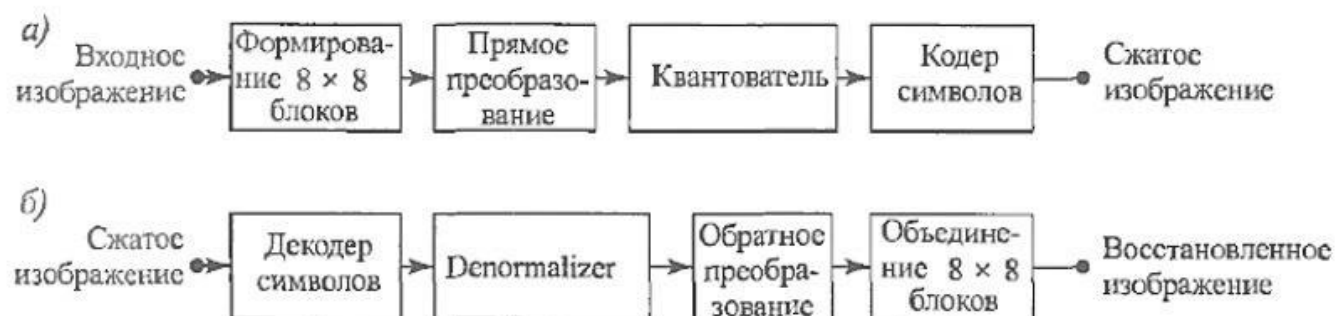


Рисунок 2.1. – а) Сжатие JPEG. б) Восстановление изображения после сжатия JPEG

В пакете IPT отсутствует функция, реализующая полностью процесс JPEG-сжатия. Однако в Matlab есть ряд функций, которые позволят в упрощенной форме рассмотреть основные этапы данного алгоритма, а именно: формирование блоков изображения, прямое преобразование и квантование.

Сжатие изображения. Первым этапом алгоритма JPEG является разделение изображения на блоки размером 8×8 пикселей. Строго размер блока

не оговаривается, но все блоки должны быть одного размера (например, возможен и размер 16×16).

На втором этапе каждый блок подвергается двумерному дискретному косинусному преобразованию (ДКП). Использование ДКП увеличивает вычислительную сложность алгоритма.

$$\text{ДКП}(i, j) = C(i) * C(j) * \sum_{x=1}^N \sum_{y=1}^N f(x, y) * \cos\left[\frac{(2x+1)i\pi}{2N}\right] * \cos\left[\frac{(2y+1)j\pi}{2N}\right],$$

$$C(i), C(j) = \begin{cases} \sqrt{\frac{1}{N}} & \text{при } i, j = 1, \\ \sqrt{\frac{2}{N}} & \text{при } i, j = 2, \dots, N. \end{cases}$$

где $f(x, y)$ – пиксел изображения, N – размер квадратного блока (в данном случае равен 8).

В результате формируется матрица частотных коэффициентов преобразования (рисунок 2.2). Коэффициент в верхнем левом углу данной матрицы называется DC-коэффициентом, остальные – AC-коэффициенты. ДКП позволяет эффективно перераспределить энергию изображения, основная часть которой содержится в DC-коэффициенте. Полученные коэффициенты преобразования также подразделяются на 3 категории:

- высокочастотные – содержат незначительную часть энергии изображения;
- среднечастотные – содержат энергию, связанную именно со спецификой сжимаемого изображения;
- низкочастотные – содержат наибольшую часть энергии изображения.

Энергия коэффициентов убывает в порядке Z сканирования коэффициентов разложения фрагмента, начиная с DC-коэффициента (рисунок 2.2).

В системе Matlab есть два способа реализации ДКП. Первый вариант заключается в использовании функции $DCT2$, которая имеет следующий синтаксис:

$$coefficients = dct2(image),$$

где $image$ – матрица элементов (в данном контексте матрица яркостей блока пикселей изображения, имеющая размеры 8×8), $coefficients$ – матрица коэффициентов преобразования.

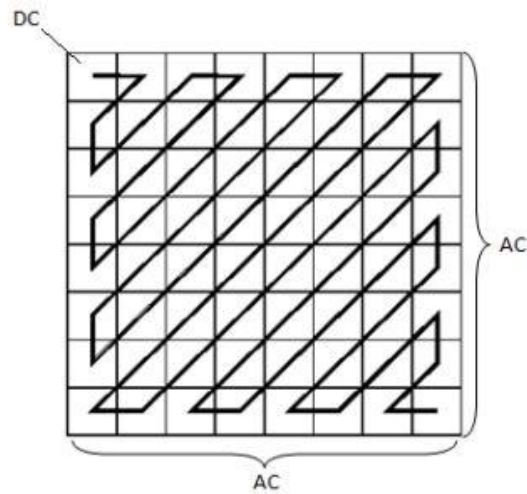


Рисунок 2.2. – Местоположение DC и AC-коэффициентов ДКП.

Пример:

```
f = [73 255 63 49 68 50 204 180;
71 170 97 65 154 49 10 24;
51 38 33 137 73 51 44 70;
66 111 219 9 76 185 143 200;
70 11 82 23 96 98 31 195;
200 162 83 36 33 202 69 140;
60 16 82 49 45 90 52 39;
56 40 96 41 29 42 153 128];
transformCoeff1 = dct2(f)
```

transformCoeff1 =

```
704.6250 -49.6535 103.6456 -33.4176 -53.1250 -73.7958 45.4189 5.9710
64.1862 47.5690 -23.5854 -32.7407 -5.9890 -85.0273 -125.2335 -7.0272
-37.9819 40.5304 17.3512 -19.4245 -29.7999 29.4565 -163.6131 11.0492
79.9618 26.4890 103.4291 52.1656 -20.6781 -65.8899 -3.3497 -44.0490
80.8750 -129.1831 100.1149 -88.0779 -52.8750 -10.0577 -4.2617 62.1714
43.6461 -41.5915 25.0799 -16.3366 -77.8999 40.6438 3.2204 -10.8930
51.0457 -17.2876 83.8869 11.9570 10.0434 42.8170 -40.1012 -71.4507
-140.7993 -45.9739 -56.1612 -20.5906 113.5466 43.8998 -32.7630 -6.3783
```

В данном примере DC-коэффициент равен 704.6250. Как видно из результатов, DC-коэффициент действительно содержит большую часть энергии данного изображения.

Существует и альтернативный вариант реализации ДКП строчно-столбцовым методом. При таком подходе прямое двумерное ДКП будет осуществляться по формуле :

$$B=C*A*C^T,$$

где - A – исходное изображение или его блок, C – матрица преобразования ДКП, C^T – транспонированная матрица преобразования ДКП.

В Matlab матрица преобразования ДКП может быть получена при помощи команды *dctmtx*, имеющей следующий синтаксис:

$$C = \text{dctmtx}(\text{dimension}) ,$$

где *dimension* – размер квадратной матрицы преобразования, C – полученная матрица преобразования ДКП.

Пример:

%В примере используем ту же матрицу f

T = dctmtx(8)

*transformCoeff2= C*f*T'*

$C =$

```
0.3536  0.3536  0.3536  0.3536  0.3536  0.3536  0.3536  0.3536
0.4904  0.4157  0.2778  0.0975 -0.0975 -0.2778 -0.4157 -0.4904
0.4619  0.1913 -0.1913 -0.4619 -0.4619 -0.1913  0.1913  0.4619
0.4157 -0.0975 -0.4904 -0.2778  0.2778  0.4904  0.0975 -0.4157
0.3536 -0.3536 -0.3536  0.3536  0.3536 -0.3536 -0.3536  0.3536
0.2778 -0.4904  0.0975  0.4157 -0.4157 -0.0975  0.4904 -0.2778
0.1913 -0.4619  0.4619 -0.1913 -0.1913  0.4619 -0.4619  0.1913
0.0975 -0.2778  0.4157 -0.4904  0.4904 -0.4157  0.2778 -0.0975
```

transformCoeff2 =

```
704.6250 -49.6535 103.6456 -33.4176 -53.1250 -73.7958 45.4189 5.9710
64.1862 47.5690 -23.5854 -32.7407 -5.9890 -85.0273 -125.2335 -7.0272
-37.9819 40.5304 17.3512 -19.4245 -29.7999 29.4565 -163.6131 11.0492
79.9618 26.4890 103.4291 52.1656 -20.6781 -65.8899 -3.3497 -44.0490
80.8750 -129.1831 100.1149 -88.0779 -52.8750 -10.0577 -4.2617 62.1714
43.6461 -41.5915 25.0799 -16.3366 -77.8999 40.6438 3.2204 -10.8930
51.0457 -17.2876 83.8869 11.9570 10.0434 42.8170 -40.1012 -71.4507
-140.7993 -45.9739 -56.1612 -20.5906 113.5466 43.8998 -32.7630 -6.3783
```

Если сравнить матрицы *transformCoeff1* и *transformCoeff2*, то очевидно, что результат полностью идентичный.

После получения матрицы коэффициентов преобразования осуществляется их квантование для уменьшения их разрядности по формуле при помощи соответствующей матрицы квантования (рисунок 2.3) [2] :

$$f'(u,v)=\text{round}\left(\frac{f(u,v)}{T(u,v)}\right),$$

где $f(u,v)$ – элемент матрицы частотных коэффициентов, полученных в результате применения ДКП; $T(u,v)$ – элемент матрицы квантования, $f'(u,v)$ – квантованный частотный коэффициент. Заметим, что при размере блока 8×8 осуществляется переход к шести битам для представления каждого коэффициента. Это позволяет своим собственным кодом представить любой из 64-х коэффициентов.

На данном этапе происходит наибольшая потеря информации. Каждый частотный коэффициент делится на соответствующий элемент матрицы квантования. В результате происходит уменьшение количества информации, вплоть до обнуления некоторых частотных коэффициентов.

3	5	7	9	11	13	15	17
5	7	9	11	13	15	17	19
7	9	11	13	15	17	19	21
9	11	13	15	17	19	21	23
11	13	15	17	19	21	23	25
13	15	17	19	21	23	25	27
15	17	19	21	23	25	27	29
17	19	21	23	25	27	29	31

Рисунок 2.3. – Матрица квантования

Пример:

%В качестве матрицы коэффициентов возьмем матрицу из предыдущего примера.

quantMatrix = [3 5 7 9 11 13 15 17;

5 7 9 11 13 15 17 19;

7 9 11 13 15 17 19 21;

9 11 13 15 17 19 21 23;

11 13 15 17 19 21 23 25;

13 15 17 19 21 23 25 27;

15 17 19 21 23 25 27 29;

17 19 21 23 25 27 29 31];

quantedCoeff = round(transformCoeff2 ./ quantMatrix)

quantedCoeff =

235 -10 15 -4 -5 -6 3 0

13 7 -3 -3 0 -6 -7 0

-5 5 2 -1 -2 2 -9 1

9 2 8 3 -1 -3 0 -2

7 -10 7 -5 -3 0 0 2

3 -3 1 -1 -4 2 0 0

3 -1 4 1 0 2 -1 -2

-8 -2 -3 -1 5 2 -1 0

В результате квантования и округления величина коэффициентов достаточно сильно изменилась. Некоторые из коэффициентов обнулились.

После того, как каждый блок частотных коэффициентов был подвергнут квантованию с помощью Z сканирования (рисунок 2.2), коэффициенты каждого блока выстраиваются в виде вектора-строки. Для дополнительного уменьшения квантования применяется сжатие без потерь (в данном случае - кодирование Хаффмана). Однако данный этап выходит за рамки данной лабораторной работы.

2.2.1 Восстановление изображения

Восстановление изображения осуществляется при помощи тех же операций, но совершаемых в обратной последовательности.

Операция обратная квантованию совершается по формуле:

$$f'(u,v) = f(u,v) * T(u,v),$$

где $f(u,v)$ – элемент матрицы частотных коэффициентов, полученных в результате применения ДКП; $T(u,v)$ – элемент матрицы квантования; $f'(u,v)$ – квантованный частотный коэффициент.

В дальнейшем для получения значения яркости пикселей применяется обратное двумерное ДКП, определяемое по формуле:

$$f(x,y) = \sum_{i=1}^N \sum_{j=1}^N C(i) * C(j) * \text{ДКП}(x,y) * \cos\left[\frac{(2x+1)i\pi}{2N}\right] * \cos\left[\frac{(2y+1)j\pi}{2N}\right],$$

$$C(i), C(j) = \begin{cases} \sqrt{\frac{1}{N}} & \text{при } i, j = 1 \\ \sqrt{\frac{2}{n}} & \text{при } i, j = 2, \dots, N \end{cases}$$

Обратное ДКП обратное ДКП выполняется строчно-столбцовым методом по формуле:

$$B = C^T * A * C$$

После данного этапа все блоки объединяются в единое изображение.

Пример:

%В качестве матрицы квантованных коэффициентов возьмем матрицу quantedCoeff

из предыдущего примера.

dequantedCoeff = quantedCoeff . quantMatrix;*

restoredImage = idct2(dequantedCoeff)

dequantedCoeff =

```

705  -50  105  -36  -55  -78  45   0
 65   49  -27  -33   0  -90 -119   0
-35   45   22  -13  -30   34 -171  21
 81   22  104   45  -17  -57   0  -46
 77 -130  105  -85  -57   0   0   50
 39  -45   17  -19  -84   46   0   0
 45  -17   76   21   0   50  -27  -58
-136  -38  -63  -23  125   54  -29   0

```


restoredImage =

```
76.4742 245.9975 68.5861 48.6337 67.1943 48.4929 207.8460 175.9226
72.7499 184.2138 89.6165 64.6029 159.1941 46.6957 3.6504 36.3835
51.3591 32.2805 43.0143 132.0959 84.7295 51.0817 38.4603 62.2768
53.8345 116.8188 213.3658 6.0500 68.0776 191.0178 140.6799 202.3203
73.7003 10.4359 79.6316 36.7039 86.3254 101.0582 23.1711 203.5607
192.5861 159.7988 85.6039 30.8471 29.5488 207.2119 70.0499 140.5297
57.2327 21.3216 81.4513 55.8806 48.7473 80.1200 57.4293 41.2360
60.6383 46.4910 89.8288 36.4148 21.8683 47.1630 157.4156 122.2810
```

Данный пример в несколько этапов выполнялся для одного блока размером 8×8 . Чтобы повторить данную операцию на всех блоках потребуется функция *blkproc*. Данная функция обеспечивает выполнение какой-либо операции на всех блоках изображения заранее заданного размера. Функция *blkproc* имеет следующий синтаксис:

$$F = \text{blkproc}(S, [m \ n], \text{fun}, P1, P2, \dots) ,$$

где *S* – исходное изображение; *m* и *n* – размерность блоков изображения, которые будут подвергаться обработке; *fun* – функция, использование которой обеспечивает обработку каждого блока; *P* – параметры функции. В представленном ниже примере вместо квантования для упрощения было осуществлено обнуление некоторых частотных коэффициентов каждого блока изображения путем наложения маски.

Пример:

```
f = imread('C:/lena.png');
f = im2double(f);
B = blkproc(f,[8 8],'dct2(x)'); %Получаем набор частотных
коэффициентов для каждого блока
mask = [1 1 1 1 0 0 0 0; %Создаем маску для обнуления частотных
коэффициентов
1 1 1 0 0 0 0 0;
1 1 0 0 0 0 0 0;
1 0 0 0 0 0 0 0;
0 0 0 0 0 0 0 0;
0 0 0 0 0 0 0 0;
0 0 0 0 0 0 0 0;
0 0 0 0 0 0 0 0];
B2 = blkproc(B,[8 8],'P1.*x',mask); %Обнуляем ряд частотных
коэффициентов, накладывая маску
f2 = blkproc(B2,[8 8],'idct2(x)'); %Восстанавливаем изображение
imshow(f)
figure, imshow(f2)
```

Подробнее рассмотрим следующую строку программного кода:

$B2 = blkproc(B, [8\ 8], 'P1.*x', mask);$

В данной строке x задает блок изображения. Данный символ является постоянным обозначением блока. Используя этот символ, блок может передаваться какой-либо функции в качестве параметра. $P1$ – параметр, которым является переменная $mask$. Используемая функция, а также местоположение параметров в выражении всегда заключается в кавычки. Например, $'P1.*x'$, что означает, что на каждый блок коэффициентов будет наложена маска.



Рисунок 2.4. – а) Оригинальное изображение; б) Сжатое изображение

Как видно из рисунка 2.4(б), после обнуления ряда коэффициентов качество изображения ухудшилось.

ПРАКТИЧЕСКАЯ ЧАСТЬ

1- Создайте функцию `compressRatio`, определяющую степень сжатия изображения;

2- Создайте функцию `directDCT`, осуществляющую прямое ДКП изображения по формуле 1;

3- Создайте функцию `invertDCT`, осуществляющую обратное ДКП изображения по формуле 5;

4- Используя изображение и предыдущий пример, осуществите 12 итераций, на каждой из которых будет происходить постепенное обнуление коэффициентов ДКП в порядке, обратному зигзаг сканированию (начиная с высокочастотного, DC-коэффициент обнулять не нужно);

5- На каждой итерации с использованием функции `compressRatio` вычислите уровень сжатия полученного изображения и с использованием функции `corr2` вычислите коэффициент корреляции Пирсона между исходным и сжатым изображением;

6- При помощи функции `plot` постройте графики зависимости а) между количеством задействованных коэффициентов и качеством изображения, б) между уровнем сжатия и качеством изображения. Оси графиков должны быть подписаны;

7- Определите приблизительное расположение высокочастотных,

среднечастотных и низкочастотных коэффициентов ДКП. Определите зависимость между коэффициентом сжатия и качеством полученного изображения;

8- Анализируя внешний вид изображений, полученных при высоких коэффициентах сжатия, определите главный недостаток формата JPEG.

6- Визуально проанализируйте наличие артефактов в полученных изображениях. Для этого выведите исходное и сжатые изображения в одно графическое окно.

7- Сохранить результаты. Прикрепить к отчёту.

Перечень контрольных вопросов

1- Что представляет собой сжатие изображений?

2- Какие методы сжатия изображений вы знаете?

3- Из каких основных этапов состоит JPEG-сжатие?

4- На каком этапе JPEG-сжатия происходит основная потеря информации?

5- Перечислить недостатки JPEG-преобразования. Что определяет RD-критерий?